

Exercícios 04 - Grafo de Fluxo de Controle (CFG)

Exercício 1 – Classificação de Pedido

1. Blocos básicos

Os blocos básicos identificados no código são:

Nó Bloco básico

- 1 Início e definição de desconto = 0
- 2 Verifica se valor ≥ 500
- 3 Define desconto = 10
- 4 Verifica se o cliente é VIP
- 5 Adiciona 5 ao desconto
- 6 Verifica se o pagamento não foi aprovado
- 7 Retorna "PAGAMENTO RECUSADO"
- 8 Calcula o valor final e retorna "PEDIDO APROVADO"
- 9 Fim

2. Decisões

O código possui três decisões principais:

1. Verificar se o valor do pedido é maior ou igual a 500.
2. Verificar se o cliente é VIP.
3. Verificar se o pagamento não foi aprovado.

Cada uma dessas decisões possui dois possíveis caminhos: verdadeiro (V) ou falso (F).

3. e 4. Grafo de Fluxo de Controle

O fluxo pode ser representado da seguinte forma:

[1] INÍCIO / desconto = 0

|
v

[2] valor ≥ 500 ?

/ \
V F

| |
v |

[3] desconto=10 |

| |
+----+----+

|
v

[4] clienteVip?

/ \
V F

| |
v |

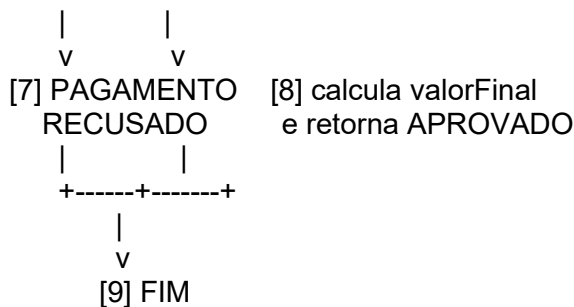
[5] desconto+=5 |

| |
+----+----+

|
v

[6] !pagamentoAprovado?

/ \
V F



O nó 7 representa uma saída antecipada. Quando o pagamento não é aprovado, o programa retorna "PAGAMENTO RECUSADO" e vai diretamente para o fim, sem realizar o cálculo do valor final.

5. e 6. Nós, arestas e complexidade ciclomática

As arestas do grafo são:

1→2; 2→3; 2→4; 3→4; 4→5; 4→6; 5→6; 6→7; 6→8; 7→9; 8→9.

Temos:

- N = 9 nós
- E = 11 arestas
-

A complexidade ciclomática é calculada pela fórmula:

$$V(G) = E - N + 2$$

Substituindo:

$$V(G) = 11 - 9 + 2 = 4$$

Também podemos conferir pelo número de decisões:

$$3 \text{ decisões} + 1 = 4$$

Portanto, os dois cálculos chegam ao mesmo resultado. A complexidade ciclomática do exercício é 4.

7. e 8. Caminhos independentes e casos de teste.

Foram escolhidos quatro caminhos independentes para representar as principais possibilidades do programa:

Caminho Fluxo	Entrada	Resultado esperado
P1 1-2F-4F-6F-8-9	valor = 100; VIP = falso; pagamento aprovado	Pedido aprovado: 100,0
P2 1-2V-3-4F-6F-8-9	valor = 500; VIP = falso; pagamento aprovado	Pedido aprovado: 450,0
P3 1-2F-4V-5-6F-8-9	valor = 100; VIP = verdadeiro; pagamento aprovado	Pedido aprovado: 95,0
P4 1-2F-4F-6V-7-9	valor = 100; VIP = falso; pagamento não aprovado	Pagamento recusado

Questões para discussão

Apesar de existirem três condições booleanas, elas poderiam gerar até $2^3 = 8$ combinações lógicas. Isso não significa que a complexidade ciclomática seja 8.

A complexidade ciclomática indica a quantidade de caminhos linearmente independentes do grafo.

Neste exercício, esse número é 4.

Também é importante observar o retorno antecipado do nó 7. Quando o pagamento não é aprovado, o sistema encerra o processamento e não chega ao cálculo do valor final.

Exercício 2 – Análise de Leituras de Temperatura

1. Blocos básicos

Os blocos básicos identificados são:

Nó Bloco básico

- 1 Início, alertas = 0 e i = 0
- 2 Verifica a condição do while: $i < \text{temperaturas.length}$
- 3 Verifica se $\text{temperaturas}[i] < 0$
- 4 Adiciona 2 aos alertas
- 5 Verifica se $\text{temperaturas}[i] > 35$
- 6 Adiciona 1 aos alertas
- 7 Incrementa i
- 8 Retorna a quantidade de alertas
- 9 Fim

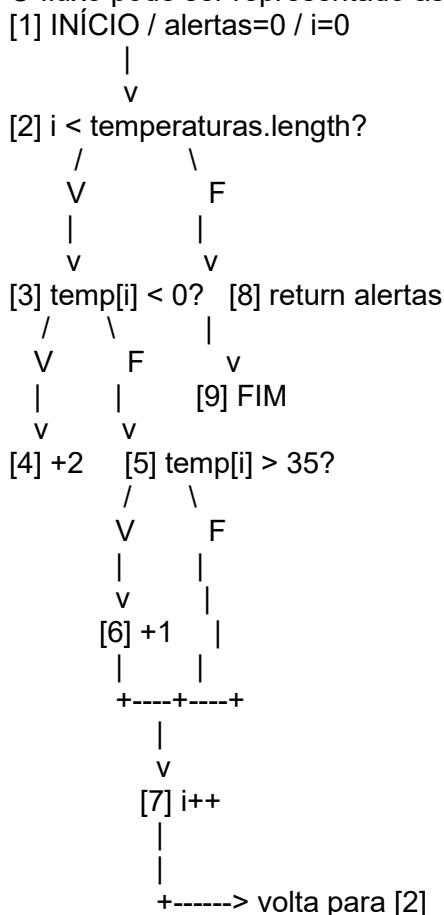
2. Decisões

Neste exercício existem três decisões:

1. A condição do while, verificando se ainda existem temperaturas para analisar.
2. A verificação se a temperatura é menor que 0.
3. A verificação se a temperatura é maior que 35.

3. e 4. Grafo de Fluxo de Controle

O fluxo pode ser representado assim:



As temperaturas podem ser classificadas em três situações:

- Menor que 0: adiciona 2 alertas.
- Maior que 35: adiciona 1 alerta.
- Entre 0 e 35, inclusive: não adiciona nenhum alerta.

5. e 6. Nós, arestas e complexidade ciclomática

As arestas são:

1→2; 2→3; 2→8; 3→4; 3→5; 4→7; 5→6; 5→7; 6→7; 7→2; 8→9.

Temos:

- N = 9 nós
- E = 11 arestas
-

Aplicando a fórmula:

$$V(G) = E - N + 2$$
$$V(G) = 11 - 9 + 2 = 4$$

Também podemos conferir pelo número de decisões:

$$3 \text{ decisões} + 1 = 4$$

Portanto, a complexidade ciclomática também é 4.

7. e 8. Caminhos independentes e casos de teste

Foram escolhidos quatro caminhos para representar as principais possibilidades do código:

Caminho	Entrada	Fluxo principal	Retorno
P1	[]	1-2F-8-9	0
P2	[-5]	1-2V-3V-4-7-2F-8-9	2
P3	[40]	1-2V-3F-5V-6-7-2F-8-9	1
P4	[20]	1-2V-3F-5F-7-2F-8-9	0

10. Por que o retorno do laço precisa aparecer no CFG?

O retorno do laço é importante porque, depois que uma temperatura é analisada e o `i` é incrementado, o programa precisa voltar para a condição do `while`.

Assim, o sistema verifica novamente se ainda existem temperaturas para serem analisadas.

Por isso, a ligação 7→2 precisa aparecer no grafo. Sem ela, o CFG não representaria corretamente o funcionamento do `while` e sua estrutura de repetição.

Questões para discussão

Um vetor pode possuir várias temperaturas e, nesse caso, algumas partes do grafo serão executadas novamente a cada repetição do while.

O vetor vazio [] é um caso importante porque permite sair do laço sem analisar nenhuma temperatura. Isso acontece porque $i = 0$ e $temperaturas.length = 0$, fazendo com que a condição do while seja falsa logo no início.

Os valores **0** e **35** também são importantes porque representam os limites das condições. O valor 0 não é menor que 0, e o valor 35 não é maior que 35. Portanto, ambos devem seguir pelo caminho que não gera alerta.

O else if também representa uma nova decisão porque existe uma segunda condição booleana que precisa ser avaliada.

Resumo Final

Exercício	Decisões	Nós (N)	Arestas (E)	V(G)	Caminhos independentes
1 – Classificação de pedido	3	9	11	4	4
2 – Temperaturas	3	9	11	4	4

Nos dois exercícios foram encontradas 3 decisões, resultando em uma complexidade ciclomática igual a 4.

Isso significa que precisamos de uma base de 4 caminhos independentes para representar as principais alternativas de execução de cada programa.

De forma geral, o CFG ajuda a visualizar como o programa pode seguir diferentes caminhos durante sua execução e facilita a criação de casos de teste para verificar esses caminhos.